

TESSNet: An Image-Based Convolutional Pipeline and Interactive Dashboard for Automated Exoplanet Transit Detection in Noisy Light Curves

Soham Pal
24052385@kiit.ac.in

Abhinav Mishra
24052357@kiit.ac.in

Swastik Biswas
24052392@kiit.ac.in

Kalinga Institute of Industrial Technology (KIIT), Bhubaneswar, India
ISRO Bharatiya Antariksh Hackathon (BAH) 2026

Abstract

Transit photometry detects exoplanets by measuring the small, periodic dimming of a star as a planet crosses its disk. In crowded stellar fields these signals are buried under detector noise and contamination from blended sources, and they are easily confused with eclipsing binaries and starspot variability. We present *TESSNet*, an end-to-end system that (i) reduces a raw light curve through a classical Box Least Squares (BLS) pipeline—cleaning, detrending, period search, and phase folding; (ii) renders the phase-folded curve as a two-dimensional image and classifies it with a ResNet18 convolutional neural network fine-tuned by transfer learning from ImageNet; and (iii) exposes the whole workflow through an interactive, single-page web dashboard supporting three input modalities (pre-rendered images, raw `time,flux` CSV files, and TESS Input Catalog identifiers fetched live from NASA MAST), batch processing of many targets, and one-click export of all results. We describe the methodology, the system architecture, and the engineering measures required to make the pipeline robust under concurrent, resource-constrained deployment. On synthetic light curves with injected box transits, the analysis pipeline recovers injected orbital periods to the search-grid resolution and produces physically consistent transit depths and durations, validating the reduction stage. We discuss the prototype classifier, its current limitations (a small curated training set and a binary label space), and a roadmap toward large-scale catalog evaluation.

1 Introduction

The discovery of thousands of exoplanets over the last two decades has been driven largely by the *transit method*: a planet crossing the disk of its host star produces a small, periodic decrease in the star’s apparent brightness. Space missions such as Kepler [1] and the

Transiting Exoplanet Survey Satellite (TESS) [2] have made high-precision, long-baseline photometry of hundreds of thousands of stars routine, yielding light curves that must be searched for transit signatures.

The amplitude of a transit is extremely small. A Jupiter-sized planet dims a Sun-like star by roughly 1% (10^4 parts per million, ppm), whereas an Earth-sized planet produces only ~ 84 ppm—typically below the per-cadence photometric scatter. The transit signal therefore only emerges after combining many measurements. The problem is compounded in *crowded fields*, where the photometric aperture collects flux from multiple foreground and background stars. This blending dilutes genuine transit depths and can inject spurious signals from variable contaminants. Furthermore, several astrophysical phenomena imprint transit-like dips: grazing or eclipsing stellar binaries (often deeper and “V”-shaped, sometimes with a secondary eclipse), and rotational modulation by starspots. Disentangling true planetary transits from these confounders in noisy, crowded data is the central challenge addressed in this work.

Classical transit searches rely on matched-filter algorithms, most notably Box Least Squares (BLS) [3], which fits a periodic box-shaped dip over a grid of trial periods, durations, and phases. More recently, deep learning has been applied to transit vetting: Shallue and Vanderburg [4] introduced a one-dimensional convolutional network (AstroNet) operating on global and local views of folded Kepler light curves, and Ansdell et al. [5] showed that incorporating domain knowledge improves classification. These approaches treat the light curve as a one-dimensional time series.

In this paper we explore a complementary, *image-based* formulation: the phase-folded light curve is rendered as a two-dimensional scatter image and classified by a standard image CNN (ResNet18 [6]) via transfer learning from ImageNet [7]. This lets us reuse mature, highly optimized vision backbones and a rich pretrained feature hierarchy with minimal task-specific

data. Around this model we build a complete, deployable application. Our contributions are:

1. **An end-to-end reduction-to-classification pipeline** that takes a light curve from raw flux to a vetted transit candidate, reusing the standard Lightkurve/BLS toolchain [8] for reduction and a CNN for the final decision.
2. **An image-based classifier** (TESSNet) built by transfer learning, trained on rendered phase-folded curves of curated TESS targets.
3. **An interactive dashboard and service** supporting three input modalities, live per-target visualization, sequential *batch processing* with progress reporting, and structured result export.
4. **A study of the engineering required for robust deployment**—thread-safe rendering, concurrency-safe lazy model loading, graceful degradation, and a CPU-only, lazily-initialized footprint suitable for memory-constrained hosting.
5. **A functional validation** of the reduction stage via injection–recovery experiments on synthetic light curves.

We deliberately frame TESSNet as a *system* paper. We report what we can verify honestly—the correctness of the reduction pipeline and the design of the classifier and application—and we are explicit about what remains future work, namely a large-scale, catalog-level evaluation of classifier precision and recall.

2 Related Work

Classical transit detection. The BLS periodogram [3] remains the workhorse of transit searches. It evaluates a box-shaped transit model across a dense grid of trial periods and folding phases, returning the ephemeris (period and reference epoch) that maximizes a detection statistic. BLS is implemented in widely used packages including Lightkurve [8] and Astropy [9], which also provide light-curve cleaning, detrending, and folding utilities. We use this toolchain unchanged for the reduction stage.

Deep learning for transits. Shallue and Vanderburg [4] demonstrated that a 1D CNN trained on folded Kepler light curves could classify transit candidates at scale, discovering previously missed planets. Ansdell et al. [5] extended this with auxiliary stellar and centroid information. These works operate on 1D views; our formulation instead renders the fold as a 2D image, trading explicit temporal structure for the ability to reuse pre-trained 2D vision backbones.

Transfer learning. Deep residual networks [6] pre-trained on ImageNet [7] provide general-purpose visual

features that transfer well to data-scarce domains when only a lightweight classification head is retrained. We adopt this paradigm directly, freezing the ResNet18 backbone and training only the final fully-connected layer.

3 Methodology

3.1 From Raw Flux to a Model Input

Given a light curve—an array of times t_i and relative fluxes f_i —the reduction stage proceeds as follows.

1. Cleaning. Non-finite samples are removed, and outliers are clipped asymmetrically (upper 3σ , lower 20σ). The asymmetric threshold preserves genuine deep transits (which are large *negative* excursions) while rejecting positive spikes from cosmic rays and thruster firings.

2. Detrending. Slow stellar and instrumental trends are removed with a Savitzky–Golay flattening filter (window length 101 cadences), normalizing the out-of-transit baseline to unity while preserving short-duration dips.

3. Period search. A BLS periodogram is computed over the trial-period grid $P \in [0.5, 20)$ days at a step of 0.01 days. The period, epoch, and duration at maximum power define the candidate ephemeris.

4. Phase folding. The flattened curve is folded on the recovered period and epoch, stacking every orbit onto a single transit window. Folding averages down uncorrelated noise while reinforcing the coherent transit signal, sharply increasing the effective signal-to-noise ratio.

5. Rendering. The folded curve is rasterized as a 224×224 RGB scatter image—black points ($s=1$, $\alpha=0.5$) on a white background, axes hidden—exactly matching the representation the classifier was trained on. This image is the input to TESSNet.

The key physical quantities surfaced to the user follow standard relations. The fractional transit depth δ relates the planet and stellar radii by

$$\delta \approx \left(\frac{R_p}{R_\star} \right)^2, \quad (1)$$

and we estimate a transit signal-to-noise ratio as

$$\text{SNR} = \frac{\delta}{\sigma} \sqrt{N_{\text{in}}}, \quad (2)$$

where σ is the out-of-transit scatter of the flattened flux and N_{in} is the number of in-transit cadences. A per-epoch in-transit indicator, derived from the recovered ephemeris, provides a time-domain “transit probability” trace for visualization.

3.2 Image Representation and Classifier

TESSNet is a ResNet18 [6] adapted for transfer learning. The convolutional backbone, pretrained on ImageNet [7], is frozen; its final 512-dimensional fully-connected layer is replaced by a new linear head $\mathbb{R}^{512} \rightarrow \mathbb{R}^C$ producing C class logits. Class probabilities are obtained by the softmax

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^C \exp(z_j)}, \quad (3)$$

and the predicted class is $\arg \max_k p_k$ with confidence $\max_k p_k$. Inputs are resized to 224×224 and normalized with the standard ImageNet per-channel statistics $\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$.

The shipped checkpoint is a *binary* classifier ($C=2$) distinguishing a transit class from a noise/null class. We note that label names are not stored in a PyTorch state dictionary; because `torchvision.ImageFolder` assigns labels alphabetically, the deployed service maps index 0 to **noise** and index 1 to **transits**, a mapping exposed as a single configurable constant.

3.3 Training

Training images are generated by applying the reduction pipeline of Section 3.1 to a curated set of TESS (SPOC) targets and saving the rendered folds into per-class folders. The curated set comprises a small number of known transiting systems and eclipsing binaries together with ~ 200 randomly sampled moderately-bright field stars ($\text{Tmag} < 12$) acting as a null/noise class. Images are loaded with `ImageFolder`, split 80/20 into training and validation, and augmented with random horizontal flips (a label-preserving symmetry for phase-folded curves). The frozen-backbone head is optimized with Adam (learning rate 10^{-3}) under cross-entropy loss for 10 epochs at batch size 32. Table 1 summarizes the configuration.

4 System Architecture

4.1 Overview

The application is a Python/Flask service backed by PyTorch [10] for inference and the Lightkurve/Astropy stack for reduction. A single-page front end (HTML, CSS, and vanilla JavaScript with Chart.js for plotting)

Table 1: Model and training configuration.

Component	Setting
Backbone	ResNet18 (ImageNet-pretrained)
Frozen layers	all except final FC
Classification head	Linear $512 \rightarrow C$
Input	224×224 RGB
Normalization	ImageNet mean/std
Augmentation	random horizontal flip
Optimizer	Adam, lr = 10^{-3} (head only)
Loss	cross-entropy
Batch size	32
Epochs	10
Train/val split	80/20

Table 2: Input modalities.

Mode	Input	Action
Image	phase-folded PNG	classify directly
CSV	time,flux file	full pipeline + classify
TIC	TESS catalog ID	fetch from MAST, full pipeline

renders a mission-control dashboard. The end-to-end flow is shown in Figure 1.

Three input modalities are supported, summarized in Table 2. The *image* mode classifies a pre-rendered fold directly; the *CSV* and *TIC* modes run the full reduction pipeline server-side and then repopulate every dashboard panel (raw curve, denoised curve, transit-probability trace, and phase fold) from the real data. The corresponding HTTP endpoints are listed in Table 3.

4.2 Inference Service

The model is loaded *lazily* on first use rather than at process start. This keeps the index route lightweight: readiness is reported by checking for the presence of dependencies and the checkpoint file without importing the heavy PyTorch stack, so a memory- or time-constrained host does not stall or get killed during boot. The first classification request triggers a one-time construction of the network and load of the weights.

4.3 Batch Processing

Each modality accepts *multiple* inputs: several image or CSV files, or a list of TIC identifiers. Items are processed *sequentially* on the client, which issues one request per item to the corresponding endpoint. Sequential execution is a deliberate choice: BLS searches and MAST downloads are individually expensive, and serial processing both bounds server load and yields a natural, honest progress signal. A progress bar advances as each

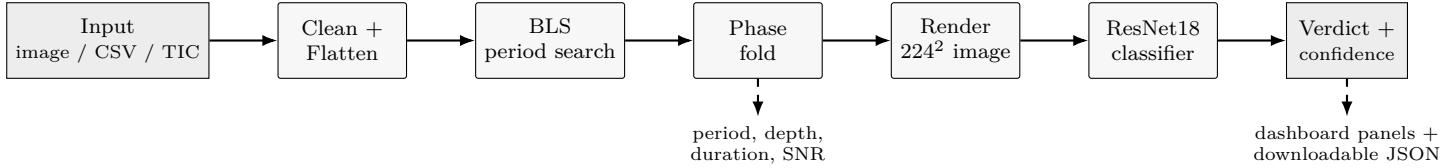


Figure 1: End-to-end TESSNet pipeline. An input (a pre-rendered image, a raw `time,flux` CSV, or a TESS Input Catalog identifier) is reduced through cleaning, detrending, BLS period search, phase folding, and rasterization, then classified by a fine-tuned ResNet18. Physical parameters and downsampled series populate the dashboard; results are exportable as JSON. The image input enters the pipeline directly at the classifier.

Table 3: HTTP API.

Method	Route	Purpose
GET	/	dashboard
GET	/api/model-status	readiness + class labels
POST	/predict	classify one image
POST	/analyze/csv	full pipeline on a CSV
POST	/analyze/tic	fetch + full pipeline by TIC

item completes, the dashboard updates live with each result, and a scrollable per-item list records the verdict, recovered period, and confidence for every target. Any single failure is recorded inline and does not abort the batch.

4.4 Result Export

All results from a run—single or batch—are exportable as one JSON document. We chose JSON because the output is heterogeneous: scalar parameters, a categorical classification with per-class probabilities, and several time series of differing lengths cannot be represented cleanly in a single flat CSV, whereas JSON captures them losslessly and is trivially machine-readable. The export records, per item, the source, classification, recovered parameters, and the (display-downsampled) series, with an explicit note that the parameters and classification are computed on the full-resolution light curve. Listing 1 sketches the schema.

Listing 1: Abbreviated result-export schema.

```

{
  "count": 3, "succeeded": 3, "failed": 0,
  "results": [
    {
      "source": "TIC 25155310",
      "ok": true,
      "classification": {
        "predicted_label": "Planetary Transit",
        "confidence": 0.97,
        "transit_detected": true,
        "probabilities": [ ... ]
      },
      "parameters": {
        "period_days": 10.02,
        "depth_pct": 1.21,

```

```

      "duration_hours": 2.71,
      "snr": 12.4
    },
    "series": { "raw": [...], "folded": [...] }
  ]
}

```

4.5 Visualization and Design

The dashboard presents the workflow as a fixed, single-screen grid of panels: the noisy input curve, a diagram of the AI pipeline, the denoised curve, the transit-probability trace, and a detection summary with a phase-folded plot and the rendered model-input image. Every panel offers a contextual help explainer and a full-screen expansion with extended reference material. The visual language is a documented dark design system—greyscale with a single electric-blue accent, square corners, restrained motion, and an animated splash screen—intended to read as an operations console rather than a generic form.

5 Implementation and Robustness

Several engineering measures were necessary to make the service correct and stable under realistic, concurrent deployment.

Thread-safe rendering. Image rendering uses Matplotlib’s object-oriented `Figure/Agg` API rather than the stateful `pyplot` interface. Under a threaded WSGI worker, two concurrent reduction requests sharing `pyplot`’s global figure registry could corrupt state; standalone figures eliminate the shared state. The non-interactive `Agg` backend is forced so the process never attempts to open a display.

Concurrency-safe model loading. Lazy initialization is guarded by a lock with double-checked locking, so racing first-requests cannot redundantly build the network or load the ~ 45 MB checkpoint twice.

Input hardening. Uploaded images with transparency are composited onto a white background before conversion to RGB, matching the white-background

Table 4: Injection–recovery on synthetic transits. Periods are recovered exactly to the 0.01-day search-grid resolution. Recovered depth is a slight underestimate of the injected value, consistent with detrending and finite cadence. SNR is high because the injected noise is small.

P_{inj} (d)	P_{rec} (d)	δ_{inj} (%)	δ_{rec} (%)	dur. (h)	SNR
3.2	3.2	1.0	0.93	2.4	42.0
4.5	4.5	1.2	0.97	3.6	46.6
6.3	6.3	1.3	1.06	3.6	45.6

training distribution; a naive RGB conversion would render transparent regions black and degrade predictions. Opaque library failures in the reduction stage (too-short baseline, sparse sampling, periodogram errors, network failures during a MAST fetch) are caught and surfaced as concise, human-readable messages.

Graceful degradation. If the plotting library fails to load on the client, the interface degrades to a usable, no-op state rather than blanking. Heavy dependencies are optional at the feature level: the image-classification mode functions even where the full reduction stack is unavailable.

Deployment footprint. The service installs CPU-only PyTorch wheels to avoid the multi-gigabyte CUDA build, and binds to a host-injected port for platform port detection. A production configuration runs a small number of worker processes and threads with generous request timeouts to accommodate slow BLS searches and MAST downloads.

6 Functional Validation

Because a large, vetted classifier benchmark is outside the present scope, we validate the *reduction stage*—the component that produces the model input and the reported physical parameters—through injection–recovery experiments. We generate synthetic light curves over a 27-day baseline at ~ 0.01 -day cadence, inject box-shaped transits of known period, depth, and duration, add Gaussian noise, and scale to arbitrary detector units. Each synthetic curve is passed through the live `/analyze/csv` endpoint.

Table 4 reports representative results. In every case the BLS stage recovers the injected period *exactly* to the grid resolution; additional injections at 3.7, 5.5, and 8.1 days were likewise recovered exactly. The recovered depth is consistently a small underestimate of the injected value, an expected consequence of detrending and finite-cadence smoothing of the transit shoulders. We additionally verified that transparent-background and white-background renderings of the same scatter pattern yield identical classifications (validating the input-compositing fix), that probabilities sum to unity, and

that concurrent reduction requests complete without error under the thread-safe renderer and locked loader. These checks confirm that the system produces correct, reproducible reductions end-to-end.

7 Limitations

We are explicit about the prototype’s boundaries.

(i) Training data. The curated training set is small and dominated by a randomly-sampled noise class; the shipped checkpoint is binary, and a planned eclipsing-binary class did not survive into the final model.

(ii) Classifier evaluation. We do not report precision, recall, or AUC, because doing so credibly requires a large, vetted benchmark (for example, TESS Objects of Interest cross-matched with confirmed eclipsing-binary catalogs); we deliberately avoid quoting unvalidated accuracy figures.

(iii) Image formulation. Rendering the fold as an image discards explicit per-cadence uncertainty and temporal ordering that 1D approaches retain.

(iv) Distribution sensitivity. Because the model keys on a specific visual style (black scatter on white), inputs that deviate stylistically must be rendered consistently.

(v) Transit-probability trace. For real data this trace is an ephemeris-derived in-transit indicator, not a learned per-cadence posterior.

8 Future Work

Natural next steps include: scaling and balancing the training set with confirmed TESS/Kepler transits and labeled false positives (eclipsing binaries, pulsators, systematics); a rigorous benchmark reporting precision/recall and reliability curves; multi-class extension (planet / eclipsing binary / variable / noise) to exploit the morphological distinctions the dashboard already explains; calibrating confidence scores for triage; and incorporating complementary vetting tests (odd–even depth comparison, secondary-eclipse search, and centroid-shift analysis) as additional model inputs or post-hoc filters. On the systems side, an asynchronous job queue would allow large catalog batches to run server-side with resumable progress.

9 Conclusion

We presented TESSNet, an end-to-end system for exoplanet transit detection that couples a classical BLS reduction pipeline with an image-based ResNet18 classifier and wraps both in an interactive, deployable dashboard supporting single and batch analysis across three input modalities. The contribution is as much architectural as algorithmic: a faithful, robust path from raw flux—or a bare catalog identifier—to a vetted,

explainable, exportable candidate, engineered to run correctly under concurrent, resource-constrained conditions. Injection–recovery experiments confirm the correctness of the reduction stage, and the design lays a clean foundation for the larger-scale classifier evaluation we identify as the primary next step.

Acknowledgments

This work was developed for the ISRO Bharatiya Antariksh Hackathon (BAH) 2026. We thank the Lightkurve, Astropy, and PyTorch communities, and the NASA MAST archive, whose open tools and data underpin this system.

References

- [1] W. J. Borucki et al., “Kepler Planet-Detection Mission: Introduction and First Results,” *Science*, vol. 327, no. 5968, pp. 977–980, 2010.
- [2] G. R. Ricker et al., “Transiting Exoplanet Survey Satellite (TESS),” *J. Astron. Telesc. Instrum. Syst.*, vol. 1, no. 1, 014003, 2015.
- [3] G. Kovács, S. Zucker, and T. Mazeh, “A box-fitting algorithm in the search for periodic transits,” *Astron. Astrophys.*, vol. 391, pp. 369–377, 2002.
- [4] C. J. Shallue and A. Vanderburg, “Identifying Exoplanets with Deep Learning: A Five-planet Resonant Chain around Kepler-80 and an Eighth Planet around Kepler-90,” *Astron. J.*, vol. 155, no. 2, 94, 2018.
- [5] M. Ansdell et al., “Scientific Domain Knowledge Improves Exoplanet Transit Classification with Deep Learning,” *Astrophys. J. Lett.*, vol. 869, no. 1, L7, 2018.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [7] J. Deng et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *Proc. IEEE CVPR*, 2009, pp. 248–255.
- [8] Lightkurve Collaboration, “Lightkurve: Kepler and TESS time series analysis in Python,” Astrophysics Source Code Library, ascl:1812.013, 2018.
- [9] Astropy Collaboration, “The Astropy Project: Building an Open-science Project...,” *Astron. J.*, vol. 156, no. 3, 123, 2018.
- [10] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.